

✓ Introduction: CFPB Consumer Complaint Database

Purpose and Scope

The **Consumer Financial Protection Bureau (CFPB)** maintains a public database of complaints submitted by consumers regarding financial products and services. Its primary purpose is to provide transparency into the marketplace, allow regulators and researchers to detect systemic issues, and ensure financial institutions respond to consumer grievances in a timely manner.

Complaints are published after companies respond or after 15 days, provided a commercial relationship is confirmed.

Complaint Categories and Available Fields

The dataset contains structured tabular metadata combined with unstructured text fields. Key metadata fields include:

- **Product & Sub-product:** Broad and specific financial product classifications (e.g., *Credit reporting, Mortgage, Debt collection, Student loans*).
- **Issue & Sub-issue:** Categorical labels detailing the specific problem (e.g., *Incorrect information on your report, Trouble during payment process*).
- **Company Information:** The targeted financial institution and whether their response was timely.
- **Geographic & Temporal Data:** `State`, `ZIP code`, and `Date received`.
- **Consumer Consent & Relational Status:** Flags indicating whether consent was provided to publish the written narrative publicly.

The Role of Complaint Narratives in NLP

The `Consumer complaint narrative` field represents raw, unstructured text written by consumers describing their experiences in their own words. Because structured categories (`Product` / `Issue`) are selected via fixed dropdowns, they don't always capture nuances or emerging pain points.

Using Natural Language Processing (NLP) on these narratives allows us to:

- Extract topics and semantic clusters beyond pre-defined categories.
- Build automated text classification models to route or tag complaints.
- Identify recurring patterns, systemic fraud, or sudden policy shifts in real time.
- Perform sentiment and emotion analysis to gauge consumer frustration levels and top keywords they used from their narrative complaints.

Project Focus: 2025 Complaints

While the full CFPB dataset dates back to 2011 and contains millions of records, this notebook focuses specifically on **complaints filed during the calendar year 2025** to perform sentiment and emotion analysis to gauge consumer frustration levels and top keywords they used from their narrative complaints..

Resources & Documentation

- **CFPB Search Interface & Data Source:** [CFPB Consumer Complaint Database](#)
- **Technical API & Field Definitions:** [CFPB API & Data Reference Dictionary](#)

✓ Load all requirements

```
#check python version
!python3 --version
```

```
Python 3.12.13
```

```
# !pip install pandas numpy
# !pip install -U spacy
# !pip install -U streamlit
# !pip install -U plotly
# !pip install -U matplotlib
# !pip install -U vaderSentiment
# !pip install -U textblob
# !pip install -U wordcloud
# !pip install -U scikit-learn
# !pip install -U sentence-transformers
# !pip install -U bertopic
# !pip install -U transformers
```

```
# !pip install -U torch
!pip install vaderSentiment
# !pip install wordcloud tqdm
```

```
Collecting vaderSentiment
  Downloading vaderSentiment-3.3.2-py2.py3-none-any.whl.metadata (572 bytes)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from vaderSentiment) (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->vaderSentimen
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->vaderSentiment) (3.18)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->vaderSentiment) (2.
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->vaderSentiment) (20
Downloading vaderSentiment-3.3.2-py2.py3-none-any.whl (125 kB)
126.0/126.0 kB 3.5 MB/s eta 0:00:00
Installing collected packages: vaderSentiment
Successfully installed vaderSentiment-3.3.2
```

```
import pandas as pd
import argparse
import sys
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from spacy.matcher import PhraseMatcher
import spacy
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from collections import Counter
```

```
pd.set_option('display.max_colwidth', None)
```

▼ Loading 2025 complaints dataset

```
def load_complaints_data(file_name, is_colab=False):
    """Loads the complaints data from either a GitHub path or Google Drive.

    Args:
        file_name (str): The relative path to the CSV file (e.g., "../../data/complaints_in_just_2025.csv").
        is_colab (bool): If True, construct path for Google Drive. Defaults to False.

    Returns:
        pandas.DataFrame: The loaded and cleaned DataFrame.
    """
    if is_colab:
        from google.colab import drive
        import os
        # Mount Google Drive
        drive.mount('/content/drive')

        # Assuming your data folder is directly under MyDrive
        gdrive_base_path = '/content/drive/MyDrive/'
        # Extract just the filename from the original file_name
        base_file_name = os.path.basename(file_name)
        full_path = os.path.join(gdrive_base_path, base_file_name)
    else:
        full_path = file_name

    df = pd.read_csv(full_path, low_memory=False)
    return df

# Load the data, setting is_colab=True since we are in a Colab environment
df = load_complaints_data("../../data/complaints_in_just_2025.csv", is_colab=True).dropna(subset=['Consumer complaint narrative'])

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
df.shape
```

```
(1221970, 16)
```

```
#df.head(5)
```

```
df.groupby('Product').size().reset_index(name='count').sort_values(
    by='count', ascending=False)
```

	Product	count	
2	Credit reporting or other personal consumer reports	912758	
3	Debt collection	100600	
5	Money transfer, virtual currency, or money service	64832	
0	Checking or savings account	51447	
1	Credit card	41494	
6	Mortgage	14015	
10	Vehicle loan or lease	11680	
9	Student loan	10881	
7	Payday loan, title loan, personal loan, or advance loan	7800	
8	Prepaid card	3822	
4	Debt or credit management	2641	

```
df.groupby([
    'Product',
    'Sub-product'
]).size().reset_index(name='count').sort_values(by='count', ascending=False)

# Count complaints for each product and sub-product combination.
# Calculate each combination's percentage of all complaints, sort by count, and keep the top 10.
product_counts = (
    df.groupby(['Product', 'Sub-product'])
      .size()
      .reset_index(name='count')
)
product_counts['percentage'] = (
    product_counts['count'] / product_counts['count'].sum() * 100
).round(2)
product_counts.sort_values(by='count', ascending=False).head(10)
```

	Product	Sub-product	count	percentage	
6	Credit reporting or other personal consumer reports	Credit reporting	906811	74.21	
24	Money transfer, virtual currency, or money service	Domestic (US) money transfer	47992	3.93	
1	Checking or savings account	Checking account	43209	3.54	
11	Debt collection	I do not know	41556	3.40	
4	Credit card	General-purpose credit card or charge card	36048	2.95	
9	Debt collection	Credit card debt	17974	1.47	
14	Debt collection	Other debt	16872	1.38	
27	Money transfer, virtual currency, or money service	Mobile or digital wallet	10748	0.88	
54	Vehicle loan or lease	Loan	10107	0.83	
51	Student loan	Federal student loan servicing	8904	0.73	

```
# The 2025 dataset has 5,443,005 rows and 16 columns
# The full dataset has 15,896,496 observations
df.shape
```

```
(1221970, 16)
```

```
# List the raw data types of all columns of the Pandas data frame
df.dtypes
```

0	
Date received	object
Product	object
Sub-product	object
Issue	object
Sub-issue	object
Consumer complaint narrative	object
Company public response	object
Company	object
State	object
ZIP code	object
Tags	object
Submitted via	object
Date sent to company	object
Company response to consumer	object
Timely response?	object
Complaint ID	int64

dtype: object

```
# Convert date columns to datetime and normalize timestamps to midnight (YYYY-MM-DD).
# pd.to_datetime handles both string and already-converted datetime values.
df['Date received'] = pd.to_datetime(
    df['Date received'], errors='coerce').dt.normalize()

df['Date sent to company'] = pd.to_datetime(
    df['Date sent to company'], errors='coerce').dt.normalize()
```

```
# Look at a few obs to make sure we have correctly formatformatted date and time.
#df.head()
```

```
# Group complaints by year received and count observations
df.groupby(df['Date received'].dt.year).size()
```

0	
Date received	
2025	1221970

dtype: int64

```
# Group complaints by product category and count observations
df.groupby(df['Product']).size().sort_values(ascending=False).reset_index(name='count')
```

	Product	count
0	Credit reporting or other personal consumer reports	912758
1	Debt collection	100600
2	Money transfer, virtual currency, or money service	64832
3	Checking or savings account	51447
4	Credit card	41494
5	Mortgage	14015
6	Vehicle loan or lease	11680
7	Student loan	10881
8	Payday loan, title loan, personal loan, or advance loan	7800
9	Prepaid card	3822
10	Debt or credit management	2641

```
# Group complaints by issue category and count observations
df.groupby('Issue').size().sort_values(ascending=False).reset_index(name='count').head(10)
```

	Issue	count
0	Incorrect information on your report	478355
1	Improper use of your report	261765
2	Problem with a company's investigation into an existing problem	168746
3	Attempts to collect debt not owed	45773
4	Other transaction problem	45405
5	Managing an account	26502
6	Written notification about debt	24189
7	False statements or representation	14029
8	Problem with a purchase shown on your statement	11096
9	Problem caused by your funds being low	9337

```
# Group by the company public response and count obsevatons
df.groupby(df['Company public response']).size().sort_values(
    ascending=False).reset_index(name='count')
```

	Company public response	count
0	Company has responded to the consumer and the CFPB and chooses not to provide a public response	622440
1	Company believes it acted appropriately as authorized by contract or law	23150
2	Company disputes the facts presented in the complaint	1105
3	Company believes the complaint provided an opportunity to answer consumer's questions	954
4	Company can't verify or dispute the facts in the complaint	508
5	Company believes complaint represents an opportunity for improvement to better serve consumers	334
6	Company believes the complaint is the result of a misunderstanding	318
7	Company believes complaint caused principally by actions of third party outside the control or direction of the company	297
8	Company believes complaint is the result of an isolated error	215
9	Company believes complaint relates to a discontinued policy or procedure	4

```
# Group by company response to consumer and count obsevatons
df.groupby(df['Company response to consumer']).size().sort_values(
    ascending=False).reset_index(name='count')
```

	Company response to consumer	count
0	Closed with explanation	768024
1	Closed with non-monetary relief	434484
2	Closed with monetary relief	15145
3	Untimely response	4201
4	In progress	116

```
# Group by timely response and Company response to consumer to consumer complaints and count obsevatons
df.groupby(['Timely response?', 'Company response to consumer']).size()
```

		0
Timely response?	Company response to consumer	
No	Closed with explanation	6063
	Closed with monetary relief	78
	Closed with non-monetary relief	325
	Untimely response	4201
Yes	Closed with explanation	761961
	Closed with monetary relief	15067
	Closed with non-monetary relief	434159
	In progress	116

dtype: int64

```
# Count up the number of unique companies having consumer complaints
df['Company'].nunique()
```

3224

```
# Find the top 10 companies with the most consumer complaints
df['Company'].value_counts().head(10)
```

	count
Company	
EQUIFAX, INC.	315504
TRANSUNION INTERMEDIATE HOLDINGS, INC.	294058
Experian Information Solutions Inc.	268568
Block, Inc.	38103
Early Warning Services, LLC	17665
CAPITAL ONE FINANCIAL CORPORATION	17160
JPMORGAN CHASE & CO.	11795
NAVY FEDERAL CREDIT UNION	10624
CITIBANK, N.A.	9160
WELLS FARGO & COMPANY	9047

dtype: int64

Summary of the data wrangling

Dataset loading and validation: Import the required Python libraries and loads the 2025 CFPB complaint file into a pandas DataFrame. Rows without a `Consumer complaint narrative` are removed.

Key insights

Credit reporting dominates the dataset. Credit reporting or other personal consumer reporting complaints account for the largest product category and substantially exceed the other categories. This concentration should be considered when interpreting NLP results because overall themes may primarily reflect credit-reporting experiences.

Debt collection and money-transfer complaints are also prominent. These categories form important secondary groups and may reveal distinct issues involving collection practices, payment transfers, and account access.

Complaint narratives are a valuable source of unstructured information. Product and issue fields provide structured labels, while the narratives provide consumers' descriptions of events, impacts, and desired resolutions. This may support text classification, topic discovery, and sentiment or outcome analysis.

Company-response fields enable outcome analysis. Comparing `Company response to consumer` with `Timely response?` can show whether timely handling is associated with different resolution outcomes. These fields describe reported responses, not necessarily whether the consumer considered the issue fully resolved.

Company counts may reflect market size as well as complaint frequency. Companies with the most complaints may serve more customers or offer more products, so complaint volume should not be interpreted as a complaint rate without an appropriate exposure denominator.

Privacy masking affects text modeling. CFPB narratives may contain redacted names, dates, account numbers, and other identifying information. These masks should be handled during preprocessing so they do not dominate the vocabulary or create misleading NLP features.

```
df_2025 = df.copy()
df_2025.rename(
    columns={'Consumer complaint narrative': 'narrative'},
    inplace=True
)
```

Double-click (or enter) to edit

```
#load modules
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
```

▼ Data preparation

The CFPB database is massive, but not all entries contain text because consumers must explicitly opt-in to share their narratives.

Here is a step-by-step pipeline to clean, preprocess, and structure this data for Natural Language Processing (NLP).

Text Preprocessing Pipeline

Consumer complaint narratives are unstructured text and may contain inconsistent capitalization, punctuation, web artifacts, privacy masks, and boilerplate language. Before applying machine-learning models such as TF-IDF with logistic regression or more advanced deep-learning methods, the narratives must be normalized to reduce noise and create a consistent vocabulary.

The preprocessing workflow includes the following steps:

- **Normalize privacy masks:** Standardize CFPB redactions such as `XXXX`, masked dates, and masked dollar amounts so that they are treated consistently rather than as unrelated tokens.
- **Convert text to lowercase:** Make words such as `Credit` and `credit` equivalent, reducing unnecessary vocabulary duplication.
- **Remove web artifacts:** Remove URLs, HTML entities, duplicate whitespace, and other formatting artifacts that do not contribute meaningful information.
- **Remove punctuation and special characters:** Reduce noise while preserving information that may be useful for interpretation.
- **Tokenize the narratives:** Split each narrative into individual words or tokens for analysis.
- **Remove stop words selectively:** Remove frequent words with limited analytical value, but retain negation terms such as `not` and `no` because they can change the meaning of a complaint.
- **Lemmatize words:** Reduce related word forms to a common base form; for example, `charged`, `charging`, and `charges` can be represented by `charge`.

- **Filter uninformative records:** Remove empty, whitespace-only, duplicate, or very short narratives that are unlikely to provide sufficient information for modeling.

This sequence produces cleaner text while preserving important complaint meaning and resolution-related language for downstream NLP analysis.

```
def normalize_masks(text):
    """Replace CFPB privacy redactions with meaningful standard tokens."""
    # Handle potential NaN values by converting to string
    if not isinstance(text, str):
        text = str(text) if text is not None else ""

    # Replace masked dates, such as XX/XX/XXXX, with a consistent date token.
    # FIXED: Added missing ']' in the quantifier {4}
    text = re.sub(r'\b[xX]{2}/[xX]{3}/[xX]{4}\b', '', text)

    # Replace masked dollar amounts or numeric values with an amount token.
    text = re.sub(r'\$[xX,]+', '', text)

    # Replace remaining sequences of two or more x/X characters with a text token.
    text = re.sub(r'[xX]{2,}', '', text)

    # Apply the standardized masking function to the complaint narratives.
    text = re.sub(r'X{2,}', '', text)

    return text

# Create a cleaned text column while preserving the original complaint narrative.
# We ensure we only process strings to avoid TypeErrors
df_2025['cleaned_narrative'] = df_2025['narrative'].fillna('').apply(normalize_masks)
```

```
# Filter out short, non-meaningful descriptions (under 20 words)
df_2025 = df_2025[df_2025['cleaned_narrative'].apply(lambda x: len(str(x).split()) > 20)]
```

```
import html

def clean_web_artifacts(text):
    text = html.unescape(text) # Converts & to &
    text = re.sub(r'https?://\S+|www.\S+', '', text) # Removes URLs
    text = re.sub(r'\s+', ' ', text).strip() # Removes duplicate whitespace/newlines
    return text
```

```
# Remove web and formatting artifacts from the cleaned complaint narratives.
df_2025['cleaned_narrative'] = df_2025['cleaned_narrative'].apply(clean_web_artifacts)
```

✓ Grouping shifting categorical targets to ensure text consistency

```
product_mapping = {
    'Credit card': 'Credit Card/Prepaid',
    'Prepaid card': 'Credit Card/Prepaid',
    'Credit card or prepaid card': 'Credit Card/Prepaid',
}
df_2025['Product_Standardized'] = df_2025['Product'].replace(product_mapping)
```

✓ Keep only rows where a commercial relationship was verified and sent to the company

```
# Keep only rows where a commercial relationship was verified and sent to the company
df_2025 = df_2025[df_2025['Submitted via'] != 'Referral']
df_2025 = df_2025.dropna(subset=['Company response to consumer'])
```

✓ Prune Document Length Extremes

Extremely long documents dilate the computational matrix size (especially for deep learning embedding vectors) and dilute the actual "sentiment" signal with thousands of words of standard chronological noise.

```
# Filter by token-count percentiles
df_2025['word_count'] = df_2025['cleaned_narrative'].apply(lambda x: len(x.split()))
```



```
min_thresh = df_2025['word_count'].quantile(0.05)
max_thresh = df_2025['word_count'].quantile(0.95)
```

```
df_filtered = df_2025[(df_2025['word_count'] >= min_thresh) & (df_2025['word_count'] <= max_thresh)]
```

```
df_filtered.shape
```

```
(1056568, 19)
```

```
# Map responses to Binary Sentiment/Outcome (1 = Relief/Positive resolution, 0 = No relief)
relief_responses = ['Closed with monetary relief', 'Closed with non-monetary relief']
df_filtered['response_binary'] = df_filtered['Company response to consumer'].apply(lambda x: 1 if x in relief_responses else 0)
```

```
/tmp/ipykernel_1551/3065730936.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
df_filtered['response_binary'] = df_filtered['Company response to consumer'].apply(lambda x: 1 if x in relief_responses else 0)
```

```
df_filtered.head()
```


	Date received	Product	Sub-product	Issue	Sub-issue	narrative	Company public response	Company	State	ZIP code	Tags
1	2025-04-08	Credit reporting or other personal consumer reports	Credit reporting	Incorrect information on your report	Account information incorrect	To Whom It May Concern, I am filing a formal complaint against Mohela, XXXX, XXXX, and XXXX for continued reporting and collection of student loan debt that is now legally void following the executive order to abolish the U.S. Department of Education. Complaint Details : 1. Illegal Credit Reporting The credit bureaus continue to report student loans that are now unverified and legally questionable due to the dissolution of the Department of Education. 2. Failure to Verify Legitimacy of Debt The loan servicer has failed to provide evidence of legal ownership, collection rights, or servicing obligations in light of the agencies abolishment. 3. Unfair and Deceptive Practices The continued collection of these debts constitutes a violation of my rights under the FCRA (15 U.S.C. 1681) and FDCPA (15 U.S.C. 1692). I demand an immediate investigation, deletion of these accounts from my credit report, and cessation of any collection efforts by my loan servicer. If this matter is not resolved promptly, I will pursue further legal action. Sincerely, XXXX XXXX	NaN	MOHELA	NE	681XX	NaN
2	2025-04-15	Student loan	Federal student loan servicing	Struggling to repay your loan	Problem with forgiveness, cancellation, or discharge	Struggling with a XXXX has significantly impacted my financial stability, making it impossible for me to meet my student loan obligations. The challenges I face daily limit my ability to work consistently or earn a sufficient income, resulting in overwhelming stress and anxiety about my financial future. Despite my efforts to seek assistance and explore repayment options, the	NaN	Maximus Federal Services, Inc.	AL	35007	NaN

burden of student loan
debt continues to loom
large. I hope for
understanding and
support from creditors
as I navigate this difficult
period, seeking
solutions that can
alleviate the pressure
and allow me to focus
on my health and well-
being.

I have taken out parent
loans on behalf of my
daughter, who is a full-
time student. I got a
notification in
XX/XX/year> that the
loans were coming due
and repayment starts in
XX/XX/year>. I called
The Department of
Education and verified
the loans are in
deferment because she
is enrolled as a full-time
student this year. They
verified this is accurate
and told me they have
no way to help me ; I
have to call Mohela
directly to fix. I called
Mohela and after
spending 3 hours on the
phone they said they
have confirmed she is a
full-time student, and
the account will be
updated. Since
XX/XX/year> they have
NOT fixed the account
and are now reporting
the account as
delinquent on my credit
as well as calling me to
collect the debt. When
they call me, if I'm
available I answer the
call, review the issue
with them, and they say
they see the notes and
will fix the account. The
last time they called me
they said they will also
transfer me to their
supervisor to discuss, in
which case the system
told me supervisors
were all busy and if I
leave my number they
will call me back within 2
days, which they did
not. I called them again
today, XX/XX/year>,
and am getting no help
from them to fix the
account. They again are
telling me they can see
the account is supposed
to be fixed but can't tell
me why it's not fixed. I
was transferred to their
Resolutions department,
which hung up on me. I
called back, they are
transferring me to their

3	2025-04-18	Student loan	Federal student loan servicing	Dealing with your lender or servicer	Trouble with how payments are being handled	NaN	MOHELA	OH	43110	Servicemember
---	------------	--------------	--------------------------------	--------------------------------------	---	-----	--------	----	-------	---------------

XXXX XXXX
Department, which I'm
now listening to a
recording telling me it's
a 3 hour wait to talk to
anyone.

I XXXX XXXX am
writing to tell you about
whats happening with
MOHELA has
inaccurately reported
negative information on
my credit file, including :
A false late payment
that has caused a
derogatory mark on my
credit report. \nA
balance that does not
reflect my actual
payment
activity.\n\nContinued
reporting of data without
my consent, in violation
of 15 U.S.C. 1641 (b) (c).\n\nFailure to provide
proper notice or
documentation
validating the
tradeline.\n\nLack of a
lawful contract or signed
authorization for data
reporting or third-party
sharing.\n\nThe account
being reported is :
Account Name :
MOHELA / Dept. of Ed
Account Number (partial) : XXXX XXXX
XXXX XXXX XXXX
Date Opened :
XXXX/2019 Past Due
Amount : {\$260.00}
Reported Balance :
{\$460.00} Payment
Terms : XXXX
XXXXmonth I previously
submitted disputes to
the major credit bureaus
under 15 U.S.C. 1681i,
but MOHELA failed to
provide full verification
or to correct or remove
the inaccurate reporting.
This is a direct violation
of FCRA provisions and
precedent such as
Cushman v. TransUnion
Corp., which requires a
creditor to verify
disputed information
with substantive
evidencenot merely
reassert the
error.\n\nFurthermore, I
assert that I am the
legal copyright holder of
my name and identity
under registration
through XXXX a lawful
notice that any use,
alteration,
misrepresentation, or
nublication of mv

7	2025-05-29	Credit reporting or other personal consumer reports	Credit reporting	Incorrect information on your report	Account information incorrect	error.\n\nFurthermore, I assert that I am the legal copyright holder of my name and identity under registration through XXXX a lawful notice that any use, alteration, misrepresentation, or nublication of mv	NaN	MOHELA	MI	48228	Servicemember
---	------------	---	------------------	--------------------------------------	-------------------------------	--	-----	--------	----	-------	---------------

permission of my
 personal data, image,
 likeness, or proprietary
 identityincluding but not
 limited to variations of
 my legal name, financial
 persona, or digital
 trailmust be explicitly
 authorized in
 writing.\n\nI have not
 granted any such
 permission to MOHELA,
 XXXX XXXX XXXX, or
 any data handling
 affiliates. Their
 continued use and
 dissemination of such
 information without my
 permission or
 contractual agreement
 constitutes a violation of
 my intellectual property
 rights, my privacy, and
 my protected
 data.\n\nAny use of my
 personal information
 without express written
 consent shall incur
 financial liability and
 legal remedy under
 applicable copyright,
 consumer protection,
 and privacy
 law.\n\nAdditionally,
 MOHELA is currently
 under investigation for
 systematic mishandling
 of servicer paperwork
 and erroneous
 reporting. I believe I
 have been directly
 affected by this, and I
 demand a full review of
 my account and
 investigation history.

In XX/XX/XXXX and
 XX/XX/XXXX my
 employer made two
 lump sum payments on
 my behalf, each in the
 amount of about
 {\$8300.00}. With each
 disbursement from my
 employer, I requested to
 be placed ahead in paid
 ahead status beginning
 on the date when
 payments resumed after
 the covid pause, which
 was XXXX or
 XXXXXXXX XXXX
 XXXXXXXX. Ive called
 Mohela and spoke to
 Mohela supervisors
 numerous times. First
 they tried to apply the
 payment based on an
 amount that would have
 been due if I wasnt on
 an income based
 repayment plan. They
 completely ignored my
 instructions and
 overcharged me. As a

```
# save the filtered DataFrame to a new CSV file
#This is the same as df_filtered.csv file
#df_filtered.to_csv('../data/complaints_2025_filtered.csv', index=False)

df_filtered.to_csv('/content/drive/MyDrive/complaints_2025_filtered.csv', index=False)
```

```
KeyboardInterrupt Traceback (most recent call last)
/tmp/ipykernel_1551/1381165240.py in <cell line: 0>()
      3 df_filtered.to_csv('Federal.../data/complaints_2025_filtered.csv', index=False)
2025-06- Student student with your how disbursements from my
10 ----> 5 df_filtered.to_csv('loan_servicing/contents/archive/MyDrive/complaints_2025_filtered.csv', index=False) NaN MOHELA MD 207XX NaN
        servicing serviced handled under the income based
        ↳ 6 frames
/usr/local/lib/python3.12/dist-packages/pandas/io/formats/csvs.py in save_chunk(self, start_i, end_i)
    322 XXXXXXXXmonth.
    323 ix = self.data_index[slicer].get_values().Thenby(*state_ifa_due)
--> 324 libwriters.write_csv_rows( to the platform migration
    325 data, my file had to be rebuilt.
    326 ix, After many calls, they
        rebuilt. I last called a
        few months ago and
        Mohela assured me that
writers.pyx in pandas._libs.writers.write_csv_rows()
```

- ✓ Sentiment analysis and Severity Score with SpaCy

```
df=df_filtered.sample(10000)
```

```
df.head()
```

paid ahead status. However, that has not happened. Im still being billed and required to

over funded. Today I

customer service no longer accepts phone calls related to billing issues. This is criminal. Its been 2 years and my request to use the lump sum payments from my employer to place me in paid ahead status has not been honored. Theyre keeping my employers money and still forcing me to pay. They are double dipping and stealing funds. Its criminal and dishonest.

	Date received	Product	Sub-product	Issue	Sub-issue	narrative	Company public response	Company	State	ZIP code	1
3179055	2025-08-18	Checking or savings account	Checking account	Problem with a lender or other company charging your account	Transaction was not authorized	On XX/XX/year>, someone had stole XXXX from my bank account and I disputed the unauthorized charges with my bank and they denied it. Then I appealed it and sent them the police report and letter from the FTC and they still denied my appeal.	NaN	FinCo Services Inc DBA Current	OH	45239	
213144	2025-01-17	Money transfer, virtual currency, or money service	Domestic (US) money transfer	Other transaction problem	NaN	I am filing a complaint against Cash App (Block , Inc.) due to inadequate customer service and unfair practices, which violate the Consumer Financial Protection Act (CFPA). Specifically, Cash App failed to take timely and effective measures to prevent and address fraud on their platform, leaving my account vulnerable and unprotected.\n\nFurthermore, their dispute resolution process was unfair and deceptive, as they did not comply with error resolution requirements under the Electronic Fund Transfer Act (EFTA) and Regulation E. These actions have caused significant inconvenience and financial loss, and I seek appropriate redress for these violations	NaN	Block, Inc.	WI	53222	
		Checking		Problem with a lender or	Transaction	I was selling sneakers on XXXX when I received a notification that I must verify my information in order to continue with the sale. A XXXX message window popped up and I entered my name address and card information, my chime card ending in XXXX. The window closed and sale went through. Then I got an alert from chime. A transaction for XXXX of cryptocurrency was made. I closed my card immediately, informed XXXX, filed a dispute with chime. Chime denied my claim even though I proved through XXXX that there was an					

4730613	2025-11-19	or savings account	Checking account	other company charging your account	transaction was not authorized	that there was an admin window opened and an admin message deleted and that XXXX is aware of these verification scams. Chime refused to contact XXXX or XXXX and wont let me file an appeal and told me the reason for the denial will be in the mail. \n\nI can prove with information provided by XXXX that I was a victim of a verification scam and chime is doing nothing. \n\nI am a single XXXX mother of XXXX kids. XXXX is too much to loose. \n\nCan you please help? I intend to get my money back, change my direct deposit and bank with a different company.	NaN	Chime Financial Inc	NJ 08723
4714587	2025-11-18	Credit reporting or other personal consumer reports	Credit reporting	Improper use of your report	Reporting company used your report improperly	The Fair Credit Reporting Act (15 U.S. Code 1681) says (1) The banking system is dependent upon fair and accurate credit reporting. Inaccurate credit reports directly impair the efficiency of the banking system, and unfair credit reporting methods undermine the public confidence which is essential to the continued functioning of the banking system. So whenever there is a violation under the FCRA, it impairs the efficiency of the banking system. This is a violation of TILA US Code 1666B since I was never late. " I recently reviewed my consumer report and noticed an incorrect late payment record. I have always maintained timely payments on this account. Under the Fair Credit Billing Act (FCBA) and the Truth in Lending Act (TILA), creditors are required to accurately report consumer transactions and provide a mechanism for correcting errors in a timely manner. Please conduct an investigation into this matter, and if you can not provide documentary evidence of these supposed late payments, I demand these payments to be updated to on time as they should be as I was NEVER late. I expect these updates to be made promptly, in compliance with the law.	Company has responded to the consumer and the CFPB and chooses not to provide a public response	TRANSUNION INTERMEDIATE HOLDINGS, INC.	GA 30296

This CFPB complaint has been filed to request pursuant to FCRA 605B (15. U. S. C.\n\n1681c2) that you, the Equifax credit reporting agency block the information appearing on my consumer credit report that is the result of identity theft, and fraud within four business days of you receiving this complaint. This fraudulent, inaccurate and

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

false information consist of personal credit card accounts, auto loans, personal loans and open end unsecured consumer accounts as follows : INQUIRIES : XXXX Date : XX/XX/2023 These accounts have resulted in several fraudulent hard inquiries and fraudulent collections appearing on my consumer FICO credit report. Ive attached a list of the hard inquiries in question as well as a copy of the consumer FICO credit report maintained by you which shows the said fraudulent items. I do not recognize these accounts, collections and hard inquiries as reported. The aforementioned accounts, collections and hard inquiries appearing on my consumer credit report maintained by you were not opened, made or initiated by me.\n\nAll of the said accounts, collections and hard inquiries are a result of identity theft, and fraud. I have attached a copy of the FTC identity theft criminal complaint case number XXXX as proof from the US federal trade commission that all of the aforementioned, consumer credit accounts, collections and hard inquiries are in fact a result of identity, theft, and fraud.

3231747	2025-08-20	Credit reporting or other personal consumer reports	Credit reporting	Incorrect information on your report	Information belongs to someone else	NaN	EQUIFAX, INC.	TX	75038
---------	------------	---	------------------	--------------------------------------	-------------------------------------	-----	---------------	----	-------

```
!python3 -m spacy download en_core_web_sm
!python3 -m spacy download en_core_web_md
!python3 -m spacy download en_core_web_lg
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en\_core\_web\_sm-3.8.0/en\_core\_web\_sm-3.8.0-py3-none-any
    12.8/12.8 MB 46.4 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
Collecting en-core-web-md==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en\_core\_web\_md-3.8.0/en\_core\_web\_md-3.8.0-py3-none-any
    33.5/33.5 MB 18.5 MB/s eta 0:00:00
Installing collected packages: en-core-web-md
Successfully installed en-core-web-md-3.8.0
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_md')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
Collecting en-core-web-lg==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en\_core\_web\_lg-3.8.0/en\_core\_web\_lg-3.8.0-py3-none-any
    400.7/400.7 MB 401.7 kB/s eta 0:00:00
Installing collected packages: en-core-web-lg
Successfully installed en-core-web-lg-3.8.0
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_lg')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

✓ Sentiment Analysis: VADER

VADER (Valence Aware Dictionary and sEntiment Reasoner) is a rule-based sentiment analysis tool designed for short, informal text. It assigns a compound sentiment score ranging from -1 to 1, where values closer to -1 indicate more negative sentiment and values closer to 1 indicate more positive sentiment. The score is calculated from the cleaned complaint narrative and is used to summarize the emotional tone of consumers' descriptions.

Because CFPB complaint narratives are often detailed, formal, and focused on specific financial problems, VADER scores should be interpreted as an indicator of expressed sentiment rather than a direct measure of complaint severity, customer satisfaction, or resolution success.

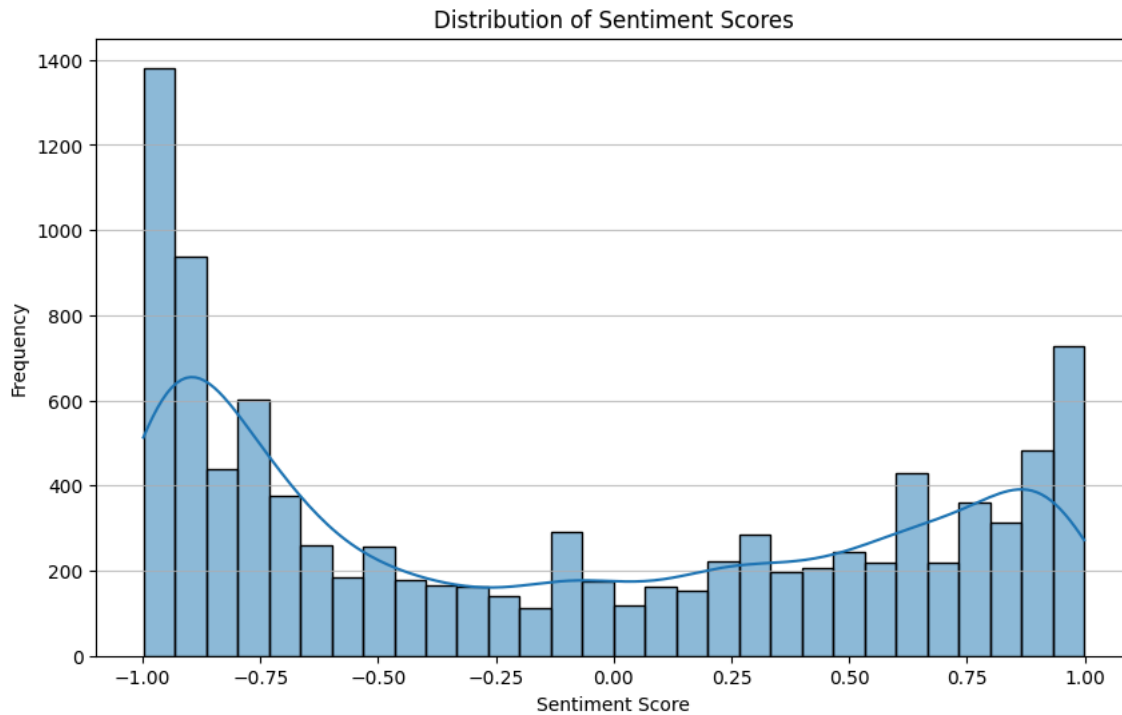
```
analyzer = SentimentIntensityAnalyzer()

def vader_score(text):
    score = analyzer.polarity_scores(text)
    return score["compound"]
```

```
df["sentiment_vader"] = df["cleaned_narrative"].apply(vader_score)
```

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.histplot(df['sentiment_vader'], bins=30, kde=True)
plt.title('Distribution of Sentiment Scores')
plt.xlabel('Sentiment Score')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.75)
plt.show()
```



✓ Interpreting the positive VADER scores

A large number of positive VADER scores does not necessarily mean that consumers were satisfied with their financial products or that their complaints were resolved positively. Several factors can explain this pattern:

- **Complaint narratives contain mixed language.** Consumers may describe a negative experience while also using positive words such as `help`, `thank`, `appreciate`, `correct`, or `resolved` when discussing a company representative, a requested outcome, or a partial improvement.
- **VADER is lexicon-based.** VADER assigns sentiment values to individual words and phrases. It may identify positive terms without fully understanding the broader financial context or the complaint's main issue.
- **Narratives often include procedural language.** Words such as `account`, `payment`, `credit`, `service`, and `information` can appear in contexts that are not emotionally positive, but the overall wording may still produce a slightly positive compound score.
- **The cleaned text changes the original writing.** Lowercasing, removing punctuation, filtering tokens, and replacing redacted information can remove emphasis, negation cues, or other context that affects sentiment scoring.
- **A positive score is not the same as a positive outcome.** VADER measures the tone expressed in the narrative; it does not measure whether the consumer received monetary relief, non-monetary relief, or a satisfactory resolution.

Therefore, VADER scores should be interpreted together with the original narrative, product and issue categories, and the company-response fields. They are most useful for comparing broad language patterns across groups rather than classifying individual complaints as resolved or unresolved.

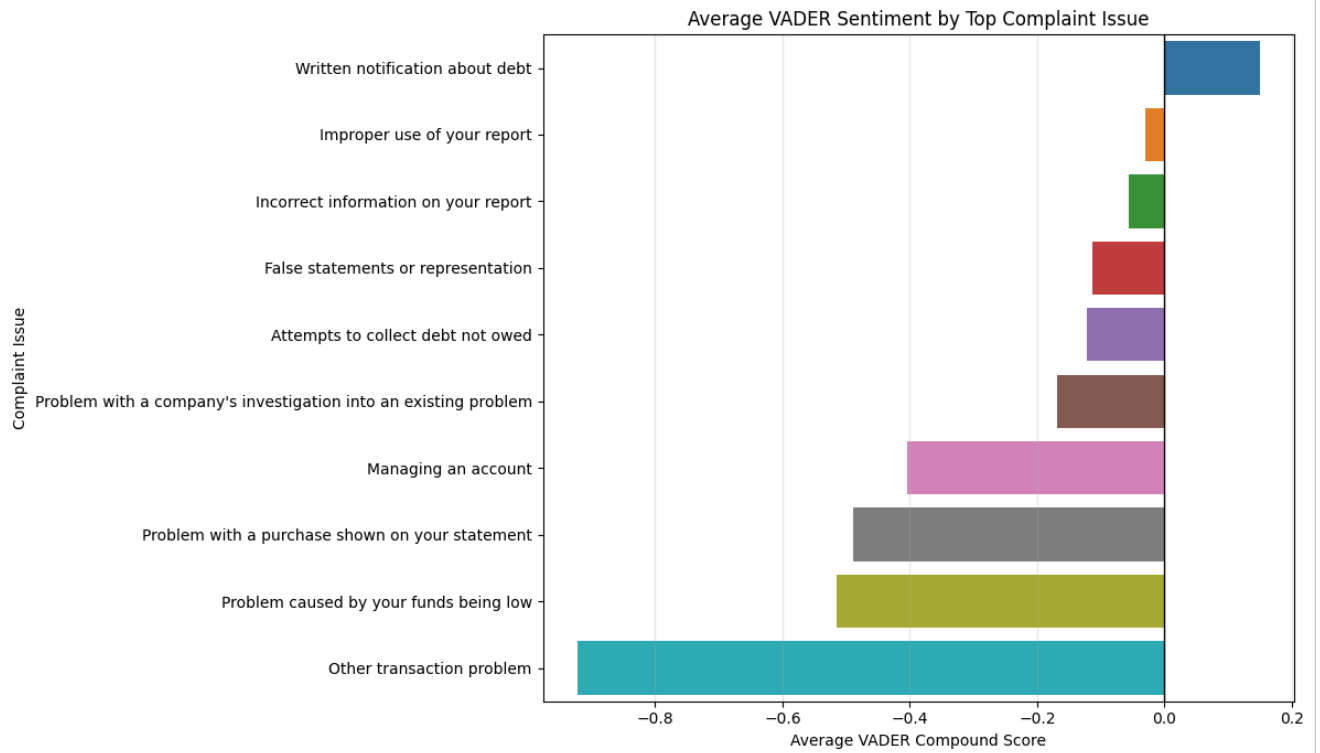
```
# Compare average VADER sentiment across the 10 most common complaint issues.
top_issues = df['Issue'].value_counts().head(10).index
issue_sentiment = (
    df[df['Issue'].isin(top_issues)]
    .groupby('Issue', as_index=False)['sentiment_vader']
    .mean()
    .sort_values('sentiment_vader', ascending=False)
)

plt.figure(figsize=(12, 7))
sns.barplot(
    data=issue_sentiment,
    x='sentiment_vader',
    y='Issue',
    hue='Issue',
    palette='tab10',
    legend=False
)
```

```

)
plt.title('Average VADER Sentiment by Top Complaint Issue')
plt.xlabel('Average VADER Compound Score')
plt.ylabel('Complaint Issue')
plt.axvline(0, color='black', linewidth=1)
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()

```



Insight for Average VADER Sentiment by Top Complaint Issue Plot

The plot 'Average VADER Sentiment by Top Complaint Issue' provides a comparison of how the emotional tone varies across different types of financial problems reported in 2025. Here is a breakdown of what the visualization reveals:

- **Relative Sentiment Comparison:** The chart ranks the top 10 issues from highest (most positive/neutral) to lowest (most negative) average compound score. A score of 0 represents neutral sentiment. Negative vs. Positive Leanings: As we noticed issues like 'Other transaction problem' or 'Managing an account' show significantly more negative scores. This typically reflects high consumer frustration regarding lost funds or inaccessible accounts.
- **The 'Neutral' Illusion:** Many issues, particularly 'False statement', might hover near or even slightly above zero. As noted, this doesn't mean the consumers are happy; rather, it often indicates the use of formal, procedural language (e.g., 'I am writing to dispute...') which VADER's general-purpose lexicon flags as neutral or slightly positive.
- **Actionable Insights:** By identifying which issues trigger the most intense negative language, regulators or companies can prioritize these areas for immediate systemic review, as they represent the highest levels of consumer distress.

Financial Severity Score

```

nlp = spacy.load("en_core_web_lg")

category_lexicon = {
    # Severity Levels
    "critical": [

```

```

"foreclosure", "eviction", "bankruptcy", "homeless", "sheriff sale",
"lease termination", "padlock", "shelter", "lawsuit", "garnishment",
"subpoena", "attorney general", "summons", "court order", "judgment",
"wage attachment", "poverty", "destitute", "hardship", "suicidal",
# Added
"default judgment", "wage garnishment", "tax levy", "asset seizure",
"vehicle repossession", "home loss", "lost my home", "forced eviction",
"utility shutoff", "shut off notice", "financial crisis",
"financial ruin", "extreme hardship", "unable to pay rent",
"unable to pay mortgage", "debt spiral", "insolvency",
"mental distress", "emotional distress", "desperate situation",
"life savings lost", "catastrophic loss", "risk of homelessness"
],

"high": [
"fraud", "identity theft", "stolen funds", "scam", "unauthorized transaction",
"account takeover", "phishing", "impersonation", "wire fraud", "forgery",
"closed account", "repossession", "collections", "frozen account",
"seized funds", "account lock", "charge off", "predatory lending", "harassment",
# Added
"unauthorized withdrawal", "unauthorized transfer", "unauthorized change",
"account hacked", "compromised account", "security breach",
"data breach", "debit card fraud", "credit card fraud",
"check fraud", "counterfeit check", "ach fraud",
"zelle fraud", "payment fraud", "synthetic identity",
"money laundering", "elder fraud", "financial abuse",
"debt collector", "aggressive collection", "collection harassment",
"wrongful repossession", "frozen funds", "restricted account",
"wrongful account closure", "claim denied", "refused investigation"
],

"medium": [
"incorrect reporting", "credit score", "dispute", "fcra", "unverified debt",
"mixed file", "late fee", "overdraft fee", "hidden charge", "unauthorized fee",
"interest rate increase", "billing error", "escrow shortage", "double charged",
# Added
"misapplied payment", "payment error", "processing error",
"accounting error", "statement error", "incorrect balance",
"duplicate charge", "duplicate transaction", "unexpected fee",
"service fee", "maintenance fee", "annual fee",
"rate increase", "payment delay", "posting delay",
"dispute denied", "dispute unresolved", "documentation error",
"customer service issue", "poor customer service",
"misleading information", "incorrect information",
"failure to respond", "communication problem",
"credit inquiry", "hard inquiry", "credit bureau error"
],

# Domain / Action Types
"legal_risk": [
"lawsuit", "attorney", "lawyer", "court", "subpoena", "legal action",
"suing", "fcra violation", "fdcpa", "tcpa", "regulatory complaint",
# Added
"class action", "arbitration", "litigation",
"legal notice", "legal proceeding", "civil action",
"consumer protection", "compliance violation",
"cease and desist", "consent order", "settlement demand",
"cfpb complaint", "occ complaint", "state regulator",
"attorney general complaint", "fair lending violation",
"respa violation", "tila violation", "regulation e",
"udap", "ecoa violation", "deceptive practices"
],

"financial_loss": [
"stolen funds", "overdraft fee", "unauthorized charge", "drained account",
"wire fraud", "double charged", "seized funds", "unauthorized fee",
# Added
"lost funds", "missing money", "missing funds",
"financial loss", "monetary loss", "out of pocket",
"refund denied", "refund not received",
"payment not credited", "withheld funds",
"negative balance", "insufficient funds",
"nsf fee", "late charge", "penalty fee",
"interest charges", "lost savings",
"lost income", "economic harm",
"unreimbursed loss", "overpayment",
"unauthorized withdrawal", "unexpected debit"
]

```

```

    ],

    "credit_damage": [
        "credit score dropped", "incorrect reporting", "derogatory mark",
        "late payment reported", "identity theft", "mixed file", "unverified debt",
        # Added
        "credit damage", "credit harm", "negative reporting",
        "inaccurate tradeline", "incorrect tradeline",
        "collection account", "charge off reported",
        "delinquency reported", "false delinquency",
        "credit report error", "credit bureau dispute",
        "hard inquiry", "unauthorized inquiry",
        "loan denial", "mortgage denial",
        "credit denial", "adverse action",
        "reduced credit limit", "account reported closed",
        "consumer reporting error", "reputation damage",
        "score decrease", "creditworthiness affected"
    ]
}

```

```

# nlp = spacy.load("en_core_web_sm")

# # Define categories and keyword lists
# category_lexicon = {
#     # Severity Levels
#     "critical": [
#         "foreclosure", "eviction", "bankruptcy", "homeless", "sheriff sale",
#         "lease termination", "padlock", "shelter", "lawsuit", "garnishment",
#         "subpoena", "attorney general", "summons", "court order", "judgment",
#         "wage attachment", "poverty", "destitute", "hardship", "suicidal"
#     ],
#     "high": [
#         "fraud", "identity theft", "stolen funds", "scam", "unauthorized transaction",
#         "account takeover", "phishing", "impersonation", "wire fraud", "forgery",
#         "closed account", "repossession", "collections", "frozen account",
#         "seized funds", "account lock", "charge off", "predatory lending", "harassment"
#     ],
#     "medium": [
#         "incorrect reporting", "credit score", "dispute", "fcra", "unverified debt",
#         "mixed file", "late fee", "overdraft fee", "hidden charge", "unauthorized fee",
#         "interest rate increase", "billing error", "escrow shortage", "double charged"
#     ],
#     # Domain / Action Types
#     "legal_risk": [
#         "lawsuit", "attorney", "lawyer", "court", "subpoena", "legal action",
#         "suing", "fcra violation", "fdcpa", "tcpa", "regulatory complaint"
#     ],
#     "financial_loss": [
#         "stolen funds", "overdraft fee", "unauthorized charge", "drained account",
#         "wire fraud", "double charged", "seized funds", "unauthorized fee"
#     ],
#     "credit_damage": [
#         "credit score dropped", "incorrect reporting", "derogatory mark",
#         "late payment reported", "identity theft", "mixed file", "unverified debt"
#     ]
# }

```

✓ Severity Score Function

```

# Initialize matcher
matcher = PhraseMatcher(nlp.vocab, attr="LOWER")

# Register each category into the matcher
for cat_name, terms in category_lexicon.items():
    patterns = [nlp.make_doc(text) for text in terms]
    matcher.add(cat_name, patterns)

```

```

def score_multi_category(doc):
    """
    Evaluates a spaCy Doc against registered categories and returns
    counts and calculated severity metrics.
    """

```



```

"""
matches = matcher(doc)

# Track term hits per category
cat_counts = {cat: 0 for cat in category_lexicon.keys()}

for match_id, start, end in matches:
    string_id = nlp.vocab.strings[match_id] # Category name
    cat_counts[string_id] += 1

# Calculate Weighted Composite Severity Score
# Critical = 5 pts | High = 3 pts | Medium = 1 pt
composite_severity = (
    (cat_counts['critical'] * 5) +
    (cat_counts['high'] * 3) +
    (cat_counts['medium'] * 1)
)

# Return dictionary of all scores
return {
    'count_critical': cat_counts['critical'],
    'count_high': cat_counts['high'],
    'count_medium': cat_counts['medium'],
    'count_legal_risk': cat_counts['legal_risk'],
    'count_financial_loss': cat_counts['financial_loss'],
    'count_credit_damage': cat_counts['credit_damage'],
    'composite_severity_score': composite_severity
}

# Batch process narratives
# Ensure 'cleaned_narrative' column contains only strings, converting any NaN to empty strings
docs = list(nlp.pipe(df['cleaned_narrative'].fillna(''), disable=["ner", "parser"]))
scores_list = [score_multi_category(doc) for doc in docs]

# Convert results into a DataFrame and combine with main data
score_df = pd.DataFrame(scores_list)
df_scored = pd.concat([df.reset_index(drop=True), score_df], axis=1)

```

```

# pd.set_option('display.max_colwidth', None)
# df_scored.query("State == 'AE'").head(5)

```

```

#df_scored.head()
state_mapping = {
    'AL': 'Alabama', 'AK': 'Alaska', 'AZ': 'Arizona', 'AR': 'Arkansas', 'CA': 'California',
    'CO': 'Colorado', 'CT': 'Connecticut', 'DE': 'Delaware', 'FL': 'Florida', 'GA': 'Georgia',
    'HI': 'Hawaii', 'ID': 'Idaho', 'IL': 'Illinois', 'IN': 'Indiana', 'IA': 'Iowa',
    'KS': 'Kansas', 'KY': 'Kentucky', 'LA': 'Louisiana', 'ME': 'Maine', 'MD': 'Maryland',
    'MA': 'Massachusetts', 'MI': 'Michigan', 'MN': 'Minnesota', 'MS': 'Mississippi', 'MO': 'Missouri',
    'MT': 'Montana', 'NE': 'Nebraska', 'NV': 'Nevada', 'NH': 'New Hampshire', 'NJ': 'New Jersey',
    'NM': 'New Mexico', 'NY': 'New York', 'NC': 'North Carolina', 'ND': 'North Dakota', 'OH': 'Ohio',
    'OK': 'Oklahoma', 'OR': 'Oregon', 'PA': 'Pennsylvania', 'RI': 'Rhode Island', 'SC': 'South Carolina',
    'SD': 'South Dakota', 'TN': 'Tennessee', 'TX': 'Texas', 'UT': 'Utah', 'VT': 'Vermont',
    'VA': 'Virginia', 'WA': 'Washington', 'WV': 'West Virginia', 'WI': 'Wisconsin', 'WY': 'Wyoming',
    'DC': 'District of Columbia', 'AS': 'American Samoa', 'GU': 'Guam', 'MP': 'Northern Mariana Islands',
    'PR': 'Puerto Rico', 'VI': 'Virgin Islands', 'AA': 'Armed Forces Americas',
    'AE': 'Armed Forces Europe', 'AP': 'Armed Forces Pacific'
}

```

```

# Top States by Severity and Complaint Volume
import plotly.express as px

# Group by State to get both the average severity and the count of complaints
state_stats = (
    df_scored
    .groupby("State")
    [ "composite_severity_score" ]
    .agg([ 'mean', 'count' ])
    .reset_index()
    .rename(columns={ 'mean': 'avg_severity', 'count': 'complaint_count' })
    .sort_values("complaint_count", ascending=True) # All states included
)

# Map to full state names using the state_mapping dictionary
state_stats['State_name'] = state_stats['State'].map(state_mapping)

```

```
# Create a bar chart: X is severity, Y is State_name (sorted by count)
fig = px.bar(
    state_stats,
    title="Average Severity Score by State (Sorted by Complaint Volume)",
    x="avg_severity",
    y="State_name",
    color="complaint_count",
    labels={
        "avg_severity": "Avg Severity Score",
        "complaint_count": "Total Complaints (Sample)",
        "State_name": "State/Territory"
    },
    hover_data=["complaint_count", "avg_severity"],
    orientation="h",
    color_continuous_scale="Viridis"
)

# Ensure all labels are shown and the order matches our sorted dataframe
fig.update_layout(
    yaxis={
        'categoryorder': 'array',
        'categoryarray': state_stats['State_name'],
        'dtick': 1
    },
    height=1200
)
fig.show()
```